

# Explanation of the Implementation of `uart_tx`

## 1 Module Objective

The file `src/uart_tx.v` implements a simple UART transmitter. Its job is to take an input byte (`tx_data`) and send it over the serial line `tx` using the classic format:

- 1 start bit with value 0
- 8 data bits
- 1 stop bit with value 1

While the byte is being transmitted, the output `tx_busy` indicates that the transmitter is busy.

## 2 Module Interface

```
module uart_tx #(
    parameter CLK_FREQ = 27_000_000,
    parameter BAUD_RATE = 115200
)(
    input wire clk,
    input wire rst_n,
    input wire tx_en,
    input wire [7:0] tx_data,
    output reg tx,
    output reg tx_busy
);
```

### Inputs

- `clk`: main system clock.
- `rst_n`: active-low reset. When it is 0, the module returns to its initial state.
- `tx_en`: pulse or enable signal that requests the start of a transmission. It is only sampled in `s_IDLE`.
- `tx_data`: byte to be transmitted.

## Outputs

- `tx`: serial output line.
- `tx_busy`: indicates whether the module is sending data.

## 3 Parameters

```
localparam CLOCKS_PER_BIT = CLK_FREQ / BAUD_RATE;
```

This constant defines how many clock cycles each UART bit should last. The important detail is that the calculation uses **integer division**; therefore, if `CLK_FREQ` is not an exact multiple of `BAUD_RATE`, the result is truncated.

For example, if:

$$\text{CLK\_FREQ} = 27\,000\,000 \quad \text{and} \quad \text{BAUD\_RATE} = 115200$$

then:

$$\text{CLOCKS\_PER\_BIT} = \left\lfloor \frac{27\,000\,000}{115200} \right\rfloor = 234$$

That means each bit remains stable for 234 `clk` cycles. With these values, the actual bit period is:

$$\frac{234}{27\,000\,000} \approx 8.667 \, \mu s$$

which corresponds to an actual baud rate of approximately:

$$\frac{27\,000\,000}{234} \approx 115384.6 \text{ baud}$$

In other words, the design does not generate exactly 115200 baud, but rather a very close approximation obtained through truncation.

## 4 State Machine

```
localparam s_IDLE = 2'b00;  
localparam s_START = 2'b01;  
localparam s_DATA = 2'b10;  
localparam s_STOP = 2'b11;
```

The module is organized as a finite-state machine with four stages:

- `s_IDLE`: waits for a new byte to transmit.
- `s_START`: sends the start bit.
- `s_DATA`: sends the 8 data bits.
- `s_STOP`: sends the stop bit.

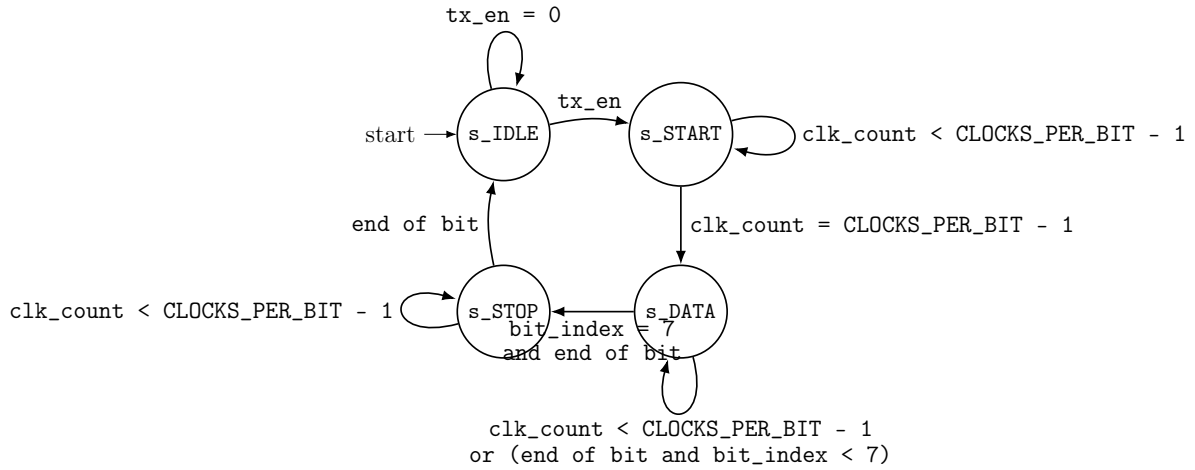


Figure 1: Visual diagram of the UART transmitter FSM.

## 5 Internal Registers

```

reg [1:0] state;
reg [15:0] clk_count;
reg [2:0] bit_index;
reg [7:0] tx_data_reg;

```

Each register has a specific function:

- **state**: stores the current state of the state machine.
- **clk\_count**: counts how many clock cycles have elapsed within the current bit.
- **bit\_index**: indicates which bit of the byte is being transmitted, from 0 to 7.
- **tx\_data\_reg**: stores a local copy of the byte being transmitted.

Storing the data in **tx\_data\_reg** is important because the input **tx\_data** could change while the transmission is in progress.

It is also worth noting that **clk\_count** is 16 bits wide. With the default parameters this is more than enough, because **CLOCKS\_PER\_BIT** = 234; however, if very high clock frequencies or very low baud rates were used, this width should be checked again.

## 6 Main Sequential Block

```

always @(posedge clk or negedge rst_n)

```

All transmitter behavior takes place inside this sequential block. That means:

- the module updates its registers on every positive edge of the clock,
- and if **rst\_n** goes low to 0, it resets immediately.

## 7 Behavior During Reset

```
if (!rst_n) begin
    state <= s_IDLE;
    clk_count <= 0;
    bit_index <= 0;
    tx_data_reg <= 8'h00;
    tx <= 1'b1;
    tx_busy <= 1'b0;
end
```

When reset occurs:

- the state returns to `s_IDLE`,
- the clock counter is cleared,
- the bit index returns to zero,
- the local data copy is cleared,
- the `tx` line is set to 1,
- and `tx_busy` is set to 0.

Keeping `tx` at 1 is correct, because in UART the line stays high when idle.

## 8 State `s_IDLE`

```
s_IDLE: begin
    tx <= 1'b1;
    clk_count <= 0;
    bit_index <= 0;

    if (tx_en) begin
        tx_data_reg <= tx_data;
        tx_busy <= 1'b1;
        state <= s_START;
    end else begin
        tx_busy <= 1'b0;
        state <= s_IDLE;
    end
end
```

This is the idle state.

- The output `tx` stays at 1, because nothing is being transmitted.
- `clk_count` and `bit_index` are reset to prepare for a new transmission.
- If `tx_en` is asserted, the module captures `tx_data`, sets `tx_busy = 1`, and moves to `s_START`.

- If `tx_en` is not asserted, the module keeps waiting.

In other words, `s_IDLE` detects the beginning of a new operation. Because everything happens inside a sequential block, detecting `tx_en` does not make `tx` go low in that same cycle: the start bit appears on the next clock edge, when the FSM is already executing `s_START`.

## 9 State `s_START`

```
s_START: begin
    tx <= 1'b0;
    if (clk_count < CLOCKS_PER_BIT - 1) begin
        clk_count <= clk_count + 1;
        state <= s_START;
    end else begin
        clk_count <= 0;
        state <= s_DATA;
    end
end
```

This is where the start bit is transmitted.

- The output `tx` is forced to 0.
- `clk_count` increments cycle by cycle.
- As long as the full bit duration has not elapsed, the state does not change.
- When the count is complete, `clk_count` is reset and the FSM moves to `s_DATA`.

The main idea is that the start bit lasts exactly one bit time measured in `clk` cycles.

## 10 State `s_DATA`

```
s_DATA: begin
    tx <= tx_data_reg[bit_index];
    if (clk_count < CLOCKS_PER_BIT - 1) begin
        clk_count <= clk_count + 1;
        state <= s_DATA;
    end else begin
        clk_count <= 0;
        if (bit_index < 7) begin
            bit_index <= bit_index + 1;
            state <= s_DATA;
        end else begin
            bit_index <= 0;
            state <= s_STOP;
        end
    end
end
```

This is the most important block, because the 8 bits of the byte are transmitted here.

## What Is Sent

The line:

```
tx <= tx_data_reg[bit_index];
```

makes the output present the bit selected by `bit_index`. Since `bit_index` starts at 0, the module sends the least significant bit (LSB) first. That behavior matches the classic UART protocol.

## How It Advances

- During one bit time, `clk_count` counts the clock cycles.
- When that time ends, the module decides whether it should move to the next bit or finish the payload.
- If `bit_index < 7`, it increments the index and remains in `s_DATA`.
- If bit 7 has already been transmitted, it resets `bit_index` and moves to `s_STOP`.

## 11 State `s_STOP`

```
s_STOP: begin
    tx <= 1'b1;
    if (clk_count < CLOCKS_PER_BIT - 1) begin
        clk_count <= clk_count + 1;
        state <= s_STOP;
    end else begin
        clk_count <= 0;
        state <= s_IDLE;
    end
end
```

In this state, the stop bit is transmitted:

- the `tx` line returns to 1,
- the module waits for one full bit time,
- and then it returns to `s_IDLE`.

When it returns to `s_IDLE`, the transmitter is ready to accept another byte. The `tx` signal remains high both during this stop bit and during idle.

## 12 Complete Transmission Sequence

If we want to transmit, for example, the byte:

$$\text{tx\_data} = 8'b1010\ 1100$$

then the order on the `tx` line will be:

1. idle: 1
2. start bit: 0
3. bit 0
4. bit 1
5. bit 2
6. bit 3
7. bit 4
8. bit 5
9. bit 6
10. bit 7
11. stop bit: 1

Since UART transmits the least significant bit first, the actual order of the data bits will be:

0, 0, 1, 1, 0, 1, 0, 1

## 13 Role of `tx_busy`

The `tx_busy` signal is asserted when the module detects `tx_en` in `s_IDLE`. Its purpose is to indicate that the transmitter is busy and should not receive a new request.

In this design, `tx_busy` is not reassigned in `s_START`, `s_DATA`, or `s_STOP`, so it keeps the value 1 throughout the entire transmission. It is only cleared on the next evaluation of `s_IDLE` when `tx_en` = 0. This makes it a simple control flag for the rest of the system, but it also shows that `tx_busy` is not wired directly to the current state; it reflects the most recently registered value.

## 14 Functional Summary

The overall behavior can be summarized as follows:

1. the module waits in `s_IDLE`,
2. when `tx_en` is asserted, it stores the byte,
3. it sends a start bit in `s_START`,
4. it sends the 8 data bits in `s_DATA`,
5. it sends a stop bit in `s_STOP`,
6. and then it returns to idle.

## 15 Key Implementation Notes

- The design uses a single state machine, which makes its flow easy to follow.
- UART timing depends directly on `CLOCKS_PER_BIT`, which here is obtained by integer division.
- Data is transmitted `LSB first`, as is standard in UART.
- There is no parity bit and no multiple stop bits; this is a basic 8N1 implementation.
- `tx_en` is only honored in `s_IDLE`; any request while `tx_busy = 1` is ignored.
- The `clk_count` counter is fixed at 16 bits, which is enough for the current configuration but not fully generic for every possible parameter combination.

## 16 Conclusion

The implementation of `uart_tx.v` is a compact and clear UART transmitter. Internally it combines:

- a state machine to organize the transmission sequence,
- a counter to define the duration of each bit,
- an index to step through the 8 bits of the byte,
- and an auxiliary register to preserve the data during the entire transmission.

From a learning perspective, this module is a good foundation for understanding serial communication, baud-rate timing, and sequential design in Verilog. At the same time, a careful reading of the RTL reveals important practical details: the actual transmission starts one cycle after detecting `tx_en`, `tx_busy` is a registered signal, and baud-rate accuracy depends on the truncation used in `CLOCKS_PER_BIT`.